
Week 3 – Progress Report

AI Agent for Supply Chain Operations

State of the Art, Frameworks & Architecture

Louis Grignola Gauthier Schmitt
Maël Tellus Matteo Schwarz

Course: SE618-1 Logistics

Institution: Sirindhorn International Institute of Technology (SIIT)

Date: February 2026

Contents

1	State of the Art: Agentic AI in Supply Chain Management	2
1.1	Overview	2
1.2	Research Paper Catalogue	2
1.3	Evaluation Methodologies in the Literature	3
1.3.1	A. Simulation-Based Evaluation (<i>Most Common</i>)	4
1.3.2	B. Real-World Deployment	4
1.3.3	C. Synthetic Scenario Evaluation	4
1.3.4	D. Comparative Framework Analysis	4
1.3.5	E. Benchmark Development (<i>Emerging</i>)	4
1.4	Our Evaluation Approach (Read-Only Agent)	4
2	Technical Frameworks & Architecture	6
2.1	Design Principle: Read-Only Access, Agentic Reasoning	6
2.2	Skills & Report Generation (SKILL.md)	6
2.3	Data Analysis & Code Execution	7
2.4	Framework Details	7
2.4.1	trpc-agent-go : Approach A, Custom Agent Engine	7
2.4.2	trpc-mcp-go : Shared MCP Integration Layer	8
2.4.3	Claude Code SDK / CLI : Approach B, AI-Native Agent	8
2.5	System Architecture	9
3	External Data Sources: Supply Chain Systems	11
3.1	Systems We Model	11
3.2	MCP Server Exposure (Read-Only)	11
3.3	Simulated Data Scale	11
3.4	Built-In Investigation Scenarios	12
3.5	Data Model	13
4	Project Timeline	14
4.1	Week-by-Week Summary	18
5	Next Steps	19
6	Bibliography	20

State of the Art: Agentic AI in Supply Chain Management

Overview

The application of agentic AI, particularly LLM-based multi-agent systems, to supply chain management (SCM) is a rapidly emerging research area. The first significant papers appeared in 2023, with a sharp acceleration in 2024–2026. While industry players such as Walmart, JD.com, SAP, and Microsoft are already deploying agentic supply chain solutions, academic research remains fragmented across computer science, operations research, and manufacturing engineering.

We reviewed **14 relevant research papers** published between 2023 and 2026. Below we catalogue them and analyze how they evaluate their approaches.

Key Insight

The field of agentic AI for supply chain is at an inflection point: industry adoption is accelerating (Gartner predicts 50% of cross-functional SC solutions will include agentic AI by 2030), but no standardized evaluation benchmarks exist yet.

Research Paper Catalogue

#	Paper	Authors	Year	Key Contribution
1	Agentic LLMs in the SC: Multi-Agent Consensus-Seeking	Jannelli et al.	2024	Multi-agent consensus framework; reduces bullwhip effect in inventory management
2	SCPA: Rethinking SC Planning	Yin et al.	2025	GenAI planning agent deployed at JD.com; +22% planning accuracy
3	Automating SC Disruption Monitoring	AlMahri et al.	2026	7 specialized agents for disruption detection; F1 0.96–0.99; 3.8 min per event
4	AIM-RM: Structured Decision Prompts & Memory Retrieval	Yoshizato et al.	2026	First LLM-MAS achieving optimal multi-echelon inventory via memory retrieval (AAMAS 2026)
5	InvAgent: LLM Multi-Agent Inventory	Quan & Liu	2024	Zero-shot multi-agent inventory management without RL fine-tuning

#	Paper	Authors	Year	Key Contribution
6	Agentic AI Sustainability Assessment	Gosmar et al.	2025	ESG assessment: 70–90% energy reduction vs. manual processes
7	Multi-Agent RAG for SC Knowledge Bases	Zhang et al.	2025	Transforms unstructured SC communications into RAG KBs (Amazon, KDD 2025)
8	KG + LLM for Warehouse Bottlenecks	Parekh et al.	2025	Knowledge graphs + LLM agent for DES warehouse bottleneck analysis
9	RAG + Prompt Engineering for SC	—	2025	Hybrid RAG for compliance monitoring and inventory anomaly detection
10	LLMs in SCM: Opportunities & Case Study	Zheng et al.	2025	Delivery delay prediction via LLM + decentralized agent system (IFAC MIM 2025)
11	OptiGuide: LLMs for SC Optimization	Li et al.	2023	LLM bridge to combinatorial optimization; deployed at Microsoft
12	MAS-ML for SC Resilience	—	2025	Multi-agent + ML framework for adaptive decision-making under disruption
13	Simulation-Driven KG + LLM for Warehouse Planning	—	2025	Human-AI collaborative warehouse planning via simulation knowledge graphs
14	SC Mapping through RAG (Electronics)	—	2026	Automated multi-tier SC mapping from SEC filings using RAG

Evaluation Methodologies in the Literature

We identified **five evaluation paradigms** across the reviewed papers. Note that many of these studies evaluate *decision-making* agents (inventory reordering, route optimization). Our project is **read-only** (we focus on retrieval, reasoning, and reporting), so we highlight which methods are relevant to our scope.

A. Simulation-Based Evaluation (*Most Common*)

Used by Papers 1, 4, 5, 12. Metrics: bullwhip effect ratio, total SC cost, stockout rate, fill rate. These measure *decision quality*, not directly applicable to our read-only agent.

B. Real-World Deployment

Used by Papers 2, 11. Metrics at JD.com: +22% planning accuracy, +2–3% stock fulfillment, –40% processing time. Relevant to us: **processing time reduction** for information retrieval.

C. Synthetic Scenario Evaluation

Used by Papers 3, 8. Metrics: F1 score (0.96–0.99), response time (3.83 min/event), cost (\$0.08/event). **Highly relevant**: this is closest to how we would evaluate our agent’s ability to correctly diagnose problems across systems.

D. Comparative Framework Analysis

Used by Papers 6, 7. Metrics: helpful answer rate (48.7% vs. 38.6% baseline), –77.4% unhelpful responses. **Directly relevant**: measures answer quality, not decision-making.

E. Benchmark Development (*Emerging*)

No standardized benchmark exists. Papers 4 and 11 propose initial protocols. This is a recognized gap in the field.

Our Evaluation Approach (Read-Only Agent)

Since our agent is **strictly read-only** (it retrieves, reasons across, and reports on supply chain data but never modifies it), our evaluation metrics focus on *information retrieval quality* and *cross-system reasoning correctness*.

Metric	Type	What It Measures
Query correctness	Accuracy	Does the agent retrieve the right data from the right system(s)?
Cross-system reasoning	Accuracy	Can it correctly connect information across SRM, WMS, OMS, TMS?
Answer completeness	Coverage	Does the response address all parts of the question?
Number of MCP calls	Efficiency	How many tool calls are needed to resolve a query?
Response time	Performance	End-to-end time from question to synthesized answer
Scenario pass rate	Correctness	Success rate on predefined investigation scenarios (Section 3.4)
Report quality	Output	Are generated reports and data analyses accurate and well-structured?

Table 2: Evaluation metrics for our read-only supply chain agent.

Research Gap Addressed

Most studies evaluate agents that *make decisions* (reorder inventory, reroute shipments). Our project addresses a complementary gap: evaluating an agent's ability to **retrieve, reason across, and report on** data from multiple siloed systems, without ever modifying it. This reflects a common real-world first step: read-only AI assistants deployed before companies trust AI with write access.

Technical Frameworks & Architecture

Design Principle: Read-Only Access, Agentic Reasoning

Our agent operates in a **controlled working environment**, a project folder where it can read supply chain data (via MCP servers), execute code for data analysis, and generate reports using skill templates. It **never writes to the supply chain databases**. All MCP tools are strictly read-only.

We explore **two approaches** for the agentic engine, both connecting to the same MCP servers:

Approach	Engine	Description
A	trpc-agent-go	Custom Go agent with full control over orchestration. Can spawn specialized sub-agents (procurement agent, inventory agent, etc.) via Graph/Chain patterns when the task is too complex for a single tool-calling loop.
B	Claude Code SDK / CLI	Leverage Claude’s built-in capabilities: autonomous reasoning, code executor (Bash, Python), sub-agent spawning , and native MCP support. Can also use SKILL.md templates for structured report generation. Works directly in a project folder.

Table 3: Two approaches for the agentic engine.

Both approaches are not mutually exclusive: we may combine them (e.g., Claude Code SDK as the user-facing layer, with trpc-agent-go sub-agents handling domain-specific reasoning).

Skills & Report Generation (SKILL.md)

Both frameworks support **skills**, reusable structured workflows defined in **SKILL.md** files. A skill is a markdown file that describes a specific capability the agent can invoke on demand.

What is a SKILL.md?

A **SKILL.md** file defines a repeatable agent workflow: its purpose, the steps to follow, expected inputs/outputs, and any constraints. When the agent (or user) invokes a skill, the agent follows the instructions in the file to produce a structured output. Think of it as a *recipe* the agent executes autonomously.

Example skills for our project:

Skill	Trigger	What It Does
/investigate-order	User asks about an order	Queries OMS, WMS, TMS, and SRM in sequence; produces a structured investigation report
/supplier-report	User asks for supplier analysis	Extracts supplier data, computes lead time and reliability stats, generates a comparison table
/inventory-analysis	User asks about stock levels	Exports inventory data to CSV, runs Python analysis, produces charts and summary
/weekly-digest	Scheduled or on-demand	Generates a weekly supply chain health report across all systems

Table 4: Example SKILL.md templates for structured agent outputs.

Data Analysis & Code Execution

The agent (in both approaches) can go beyond simple Q&A by leveraging a **code executor**:

- **Extract data** from MCP servers into CSV/Excel files in the working folder
- **Run Python/R scripts** to perform statistical analysis, compute KPIs, detect anomalies
- **Generate visualizations** (charts, tables) as part of structured reports
- **Cross-reference datasets** programmatically when the reasoning requires joining data from multiple systems

This means the agent can answer not just “What is the stock level of SKU-456?” but also “Analyze delivery performance trends over the last 3 months and flag carriers with declining on-time rates”, by extracting the data, writing an analysis script, and producing a report with charts.

Framework Details

trpc-agent-go : Approach A, Custom Agent Engine

Repository: github.com/trpc-group/trpc-agent-go

Go 1.21+ | Apache 2.0

A production-grade Go framework for building LLM-powered agents. Our team is **actively contributing** to this project.

Feature	Description
Multi-Agent Orchestration	Chain (sequential), Parallel (concurrent), Graph (conditional routing)
Tool Integration	FunctionTool, MCPTool (connects to our MCP servers), SkillTool
Sub-Agent Spawning	Spawn specialized agents when a single tool-calling loop is insufficient
Memory & Persistence	Persistent state across sessions for long-running investigations
Streaming	Real-time token streaming, SSE-compatible for web frontends
Observability	Langfuse integration for tracing and evaluation

When to use sub-agents: If a query like “Why is order #123 delayed?” requires checking 4 systems with interdependent logic, a single tool-calling agent may struggle. With trpc-agent-go,

we can spawn a *procurement sub-agent*, an *inventory sub-agent*, etc., each focused on one domain, then have an orchestrator synthesize their findings.

trpc-mcp-go : Shared MCP Integration Layer

Repository: github.com/trpc-group/trpc-mcp-go

Go | MCP 2025-03-26

A Go implementation of the **Model Context Protocol** (MCP), the open standard for agent-to-tool communication. Used by **both approaches**.

Feature	Description
Full MCP Specification	Complete compliance with MCP protocol (2025-03-26)
Tool Framework	Register, discover, and execute tools with structured parameters
Read-Only Enforcement	All tools exposed are strictly read-only (SELECT queries only)
Transport	Streamable HTTP (recommended), STDIO, SSE
Progress Notifications	Real-time updates for long-running queries

Claude Code SDK / CLI : Approach B, AI-Native Agent

Documentation: [Anthropic Platform](#)

TypeScript, Python

Anthropic’s open-source framework exposing the same infrastructure behind Claude Code. Can also be used directly via the `claude` CLI.

Feature	Description
Built-in Tools	Read, Write, Bash (code executor), Glob, Grep, WebSearch
Sub-Agent Spawning	Spawn focused sub-agents with isolated context for parallel investigation
MCP Integration	Native connection to any MCP server (our 4 SC servers)
SKILL.md Support	Define reusable workflows as skill files the agent can invoke
Session Persistence	Resume investigations across sessions with full context
Permissions	Fine-grained access control; enforce read-only mode on SC data
Working Folder	Operates within a project directory: exports, reports, scripts all live there

System Architecture

The following diagram shows the complete system: the user interacts with either Approach A or B, both of which connect to the same four read-only MCP servers. The agent operates in a controlled working environment where it can generate reports, extract data for analysis, and write outputs.

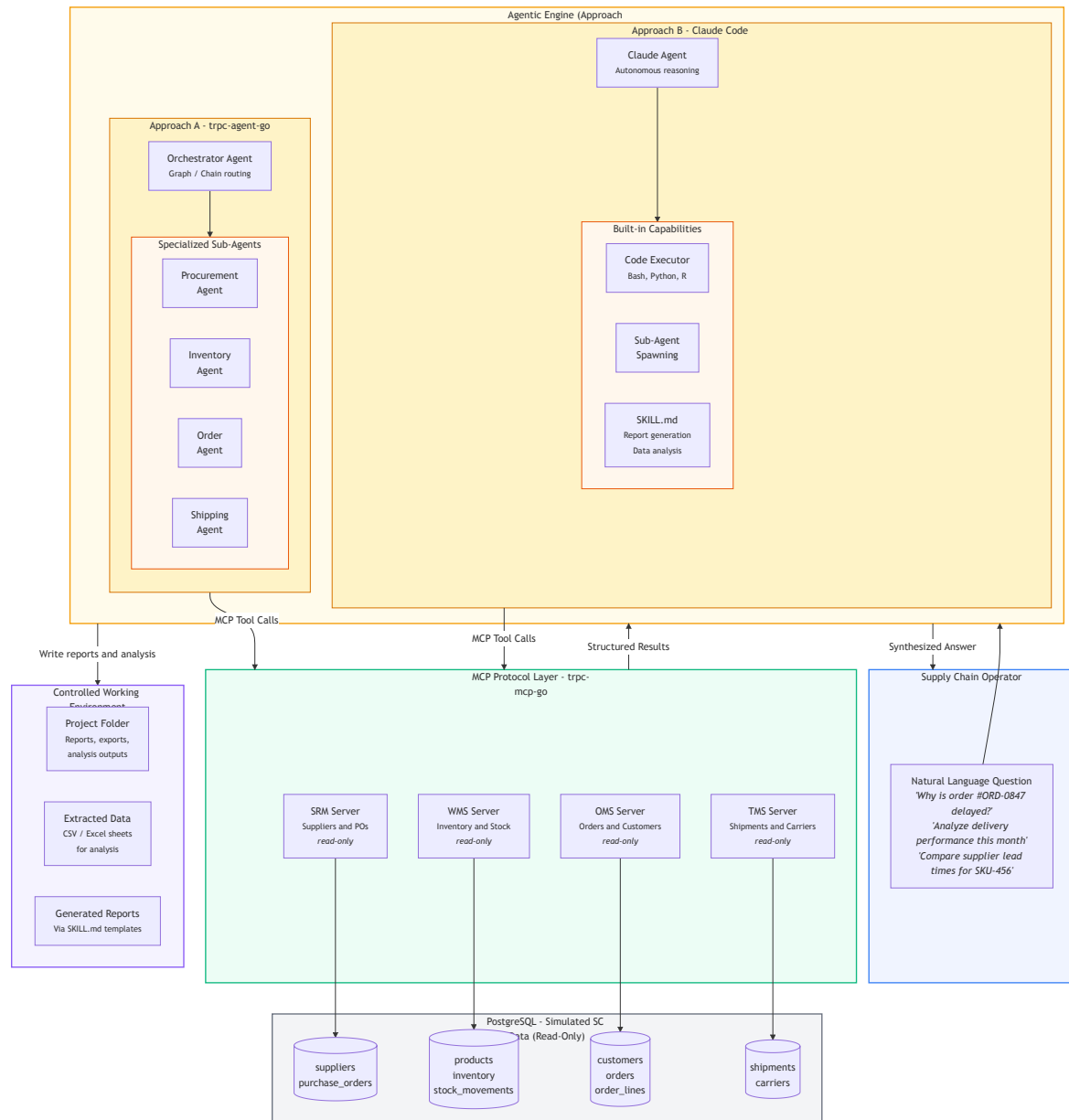


Figure 1: End-to-end architecture: two agentic approaches (trpc-agent-go or Claude Code SDK/CLI) connecting to four read-only MCP servers, with a controlled working environment for reports and data analysis.

External Data Sources: Supply Chain Systems

Systems We Model

We simulate a **simplified distribution company** with four enterprise systems. These are the external data sources our agent queries through MCP. **All access is read-only.**

System	Full Name	Real-World Examples	What It Manages
SRM	Supplier Relationship Mgmt	SAP Ariba, Coupa	Suppliers, Purchase Orders, Lead Times, Performance
WMS	Warehouse Management	SAP EWM, Manhattan	Products, Inventory, Stock Movements, Locations
OMS	Order Management	SAP SD, Salesforce	Customers, Sales Orders, Order Lines, Fulfillment
TMS	Transport Management	SAP TM, project44	Shipments, Carriers, Tracking, Route Planning

MCP Server Exposure (Read-Only)

Each system is implemented as a **separate MCP server** with read-only tools:

MCP Server	Read-Only Tools	Example Query
<code>srm-server</code>	<code>get_supplier</code> , <code>list_purchase_orders</code> , <code>get_supplier_perf</code>	“Which suppliers deliver product X?”
<code>wms-server</code>	<code>check_inventory</code> , <code>get_stock_movements</code> , <code>get_product</code>	“Current stock of SKU-456?”
<code>oms-server</code>	<code>get_order</code> , <code>list_orders_by_customer</code> , <code>get_order_lines</code>	“All pending urgent orders?”
<code>tms-server</code>	<code>get_shipment</code> , <code>get_carrier_performance</code> , <code>track_delivery</code>	“Where is the shipment for order #123?”

Simulated Data Scale

Entity	Count	Notes
Suppliers	5	Different countries, varying reliability
Products	30	3 categories, fast/slow movers
Customers	50	Retail, wholesale, enterprise segments
Orders	500	3 months of transactional history
Purchase Orders	100	From various suppliers
Shipments	400	Including exception statuses
Inventory Records	60	30 products × 2 warehouses

Built-In Investigation Scenarios

We embed five realistic scenarios in the seed data. These serve both as demonstrations and as **evaluation benchmarks**: we know the correct answer for each and can measure the agent’s ability to find it.

1. **Stockout Investigation:** Product has 0 inventory; orders unfulfilled; purchase order delayed from supplier. Agent must connect WMS → OMS → SRM.
2. **Late Delivery:** Urgent customer order; shipment in “exception” status; carrier with poor on-time rate. Agent must connect OMS → TMS.
3. **Supplier Comparison:** Same product from 2 suppliers with different lead times, costs, reliability. Agent must query SRM and cross-reference with WMS receiving history.
4. **Demand Spike:** Sudden order surge depleting inventory faster than the reorder point predicted. Agent must analyze OMS trends and WMS levels.
5. **Cross-Warehouse Transfer:** Product available in WH-B but customer is near WH-A. Agent must check WMS for both warehouses and TMS for shipping options.

Data Model

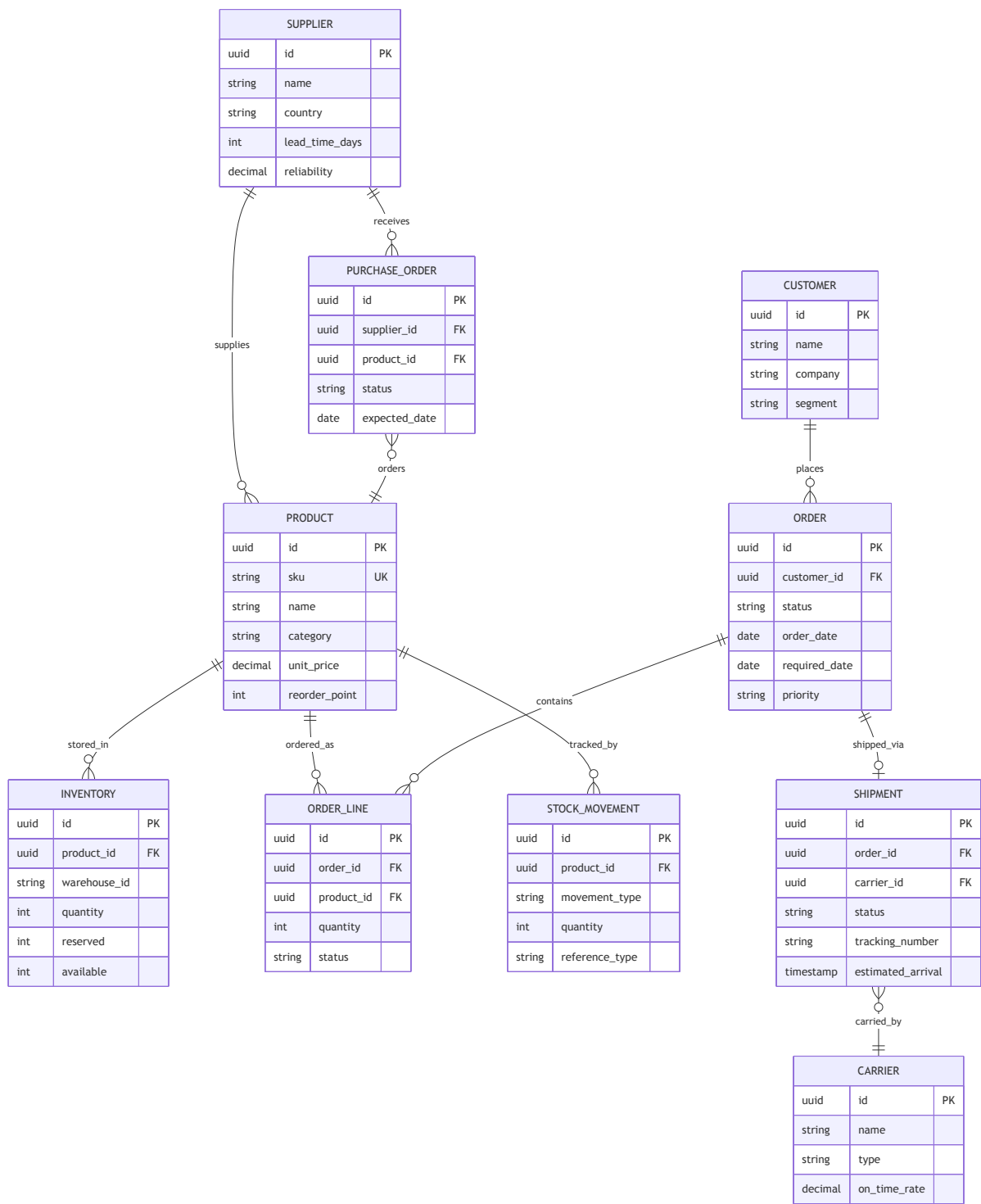
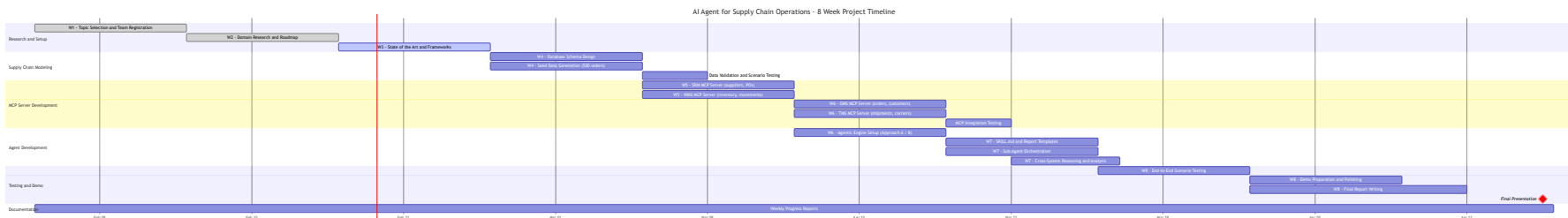


Figure 2: Entity-relationship diagram: cross-system data model spanning SRM, WMS, OMS, and TMS.

Project Timeline

The project spans **8 weeks** (February 5 – April 15, 2026), organized into five parallel workstreams.



Week-by-Week Summary

Week	Dates	Focus	Deliverable
1	Feb 5–11	Topic Selection & Registration	Project proposal
2	Feb 12–18	Domain Research & Roadmap	Progress report, roadmap
3	Feb 19–25	State of the Art & Frameworks	Literature review, framework analysis
4	Feb 26–Mar 4	Database Schema & Data Gen.	PostgreSQL schema, seed scripts
5	Mar 5–11	MCP Servers (SRM, WMS)	Working read-only MCP servers for 2 systems
6	Mar 12–18	MCP Servers (OMS, TMS) + Agent	All 4 MCP servers, agent scaffolding, skills
7	Mar 19–25	Agent Integration & Reasoning	Working agent, sub-agents, data analysis
8	Mar 26–Apr 15	Testing, Demo & Final Report	Scenario evaluation, presentation, report

Next Steps

- **This week (W3):** Finalize literature review; validate framework selection with tutor.
- **Next week (W4):** Begin PostgreSQL schema implementation for all four systems; write seed data generation scripts with embedded investigation scenarios.
- **Ongoing:** Continue contributions to the `trpc-agent-go` open-source project.

Bibliography

- [1] Jannelli, V., Schoepf, S., Bickel, M., Netland, T., & Brintrup, A. (2024). Agentic LLMs in the Supply Chain: Towards Autonomous Multi-Agent Consensus-Seeking. *arXiv:2411.10184*. Published in *Int. J. Production Research*, 2025. [arXiv](#)
- [2] Yin, J. et al. (2025). Rethinking Supply Chain Planning: A Generative Paradigm. *arXiv:2509.03811*. [arXiv](#)
- [3] AlMahri, S., Xu, L., & Brintrup, A. (2026). Automating Supply Chain Disruption Monitoring via an Agentic AI Approach. *arXiv:2601.09680*. [arXiv](#)
- [4] Yoshizato, K. et al. (2026). AI Agent Systems for Supply Chains: Structured Decision Prompts and Memory Retrieval. *arXiv:2602.05524*. Accepted at AAMAS 2026. [arXiv](#)
- [5] Quan, Y. & Liu, Z. (2024). InvAgent: A LLM-based Multi-Agent System for Inventory Management. *arXiv:2407.11384*. [arXiv](#)
- [6] Gosmar, D. et al. (2025). Agentic AI Sustainability Assessment for SC Document Insights. *arXiv:2511.07097*. [arXiv](#)
- [7] Zhang, Y. et al. (2025). From Unstructured Communication to Intelligent RAG: Multi-Agent Automation for Supply Chain Knowledge Bases. *arXiv:2506.17484*. KDD 2025 Workshop. [arXiv](#)
- [8] Parekh, R. et al. (2025). Leveraging Knowledge Graphs and LLM Reasoning to Identify Operational Bottlenecks for Warehouse Planning. *arXiv:2507.17273*. [arXiv](#)
- [9] Integrating RAG with Prompt Engineering for Knowledge-Driven SC Solutions. (2025). *ResearchGate Publication 393479215*. [ResearchGate](#)
- [10] Zheng, G. et al. (2025). LLMs in SCM: Opportunities and a Case Study. *IFAC-PapersOnLine*, MIM 2025. [ScienceDirect](#)
- [11] Li, B. et al. (2023). Large Language Models for Supply Chain Optimization (OptiGuide). *arXiv:2307.03875*. [arXiv](#)
- [12] Enhancing SC Resilience with Multi-Agent Systems and Machine Learning. (2025). *Am. J. Eng. & Tech. Journal*
- [13] Intelligent Human-Machine Partnership for Manufacturing: Simulation-Driven KGs and LLM Collaboration. (2025). *arXiv:2512.18265*. [arXiv](#)
- [14] Supply Chain Mapping through RAG: Applications to the Electronics Industry. (2026). *J. Operational Research Society*. [Taylor & Francis](#)